PROGRAMACIÓN IV

Trabajo Práctico - CRUD Categorías + CRUD Productos + CRUD Ingredientes

OBJETIVO GENERAL

Desarrollar una aplicación Full Stack conectando un frontend React + TypeScript con un backend FastAPI. El estudiante implementará el CRUD completo de Categorías, Productos e ingredientes desde la interfaz de usuario, integrando el backend mediante fetch nativo. Los CRUDs de Pedido y DetallePedido serán construidos en el backend y demostrados mediante un archivo REST Client.

MARCO TEÓRICO

Concepto	Aplicación en el proyecto
React	Biblioteca para construir interfaces de usuario mediante componentes reutilizables.
TypeScript	Superset tipado de JavaScript para detectar errores en compilación y mejorar la legibilidad.
Vite	Herramienta de build moderna que inicializa proyectos React con TypeScript, reemplazando Create React App.
FastAPI	Framework Python de alto rendimiento para construir APIs REST. Permite definir endpoints, validar datos con Pydantic y exponer documentación automática en /docs.
Pydantic	Librería de validación de datos usada por FastAPI para definir modelos (schemas) con tipado estricto.
fetch nativo	API del navegador para realizar peticiones HTTP. Se usa con async/await para consumir la API REST desde el frontend.
tanStack form	Librería para formularios fuertemente tipados y escalables
tanStack table	Librería para el manejo de tablas
Interface	Construcción de TypeScript que define la forma de un objeto (ej: Categoria, Producto).
Tailwind CSS	Framework de CSS utilitario. Estilos aplicados directamente en el JSX mediante clases predefinidas.
CORS	Mecanismo que permite al frontend (localhost:5173) comunicarse con el backend (localhost:8000) en dominios distintos.

CASO PRÁCTICO

Parte A — Backend con Fast Api

Tomando como base el trabajo práctico de la unidad 7:

a) Crear y activar un entorno virtual Python, crea archivo de dependencias e instalar las dependencias:

```
python -m venv .venv
source .venv/bin/activate   # macOS/Linux
.venv\Scripts\activate     # Windows
pip install -r requirements.txt
```

b) Definir los models y schemas del módulo Ventas, Pagos & Trazabilidad.
(SVG adjunto a este pdf)

c) Crear un CRUD siguiendo la arquitectura **Models-Schemas-Repository-UoW-Service-Router**

Notas:

Recuerda agregar CORS para poder consumir tu API desde su frontend.

Estructura de ejemplo:

```
app/
├── core/
│   ├── config.py
│   ├── database.py
│   ├── repository.py
│   └── unit_of_work.py
│
├── modules/
│   ├── health/
│   │   └── router.py
│   │
│   ├── heroes/
│   │   ├── models.py
│   │   ├── repository.py
│   │   ├── router.py
│   │   ├── schemas.py
│   │   ├── service.py
│   │   └── unit_of_work.py
│   │
│   └── teams/
│   ├── models.py
```

```
│   ├── repository.py
│   ├── router.py
│   ├── schemas.py
│   ├── service.py
│   └── unit_of_work.py
│
├── .env
├── main.py
├── README.md
├── requirements.txt
└── seed.py
```

d) La solución deberá de implementar todo lo aprendido en unidades anteriores.

e) Ejecutar el BackEnd:

```
python -m fastapi dev main.py
```

Parte B — Frontend con React + TypeScript (MEJORADO)

f) Páginas (Routing)

La aplicación debe estar organizada en páginas separadas.

Crear en src/pages/:

- ***CategoriesPage.tsx***
- ***ingredientsPage.tsx***
- ***ProductsPage.tsx***

CategoriesPage

Debe contener:

- *Toda la lógica de categorías:*
  - *useQuery*
  - *useMutation*
  - *manejo del modal*
  - *tanstack form*
  - *tanstack table*
- *Renderiza:*
  - *CategoriaList*
  - *CategoriaModal*
  - *botón “Nueva categoría”*

ProductsPage

- *Estructura similar a CategoriasPage*
- *Preparada para trabajar con productos*
- *Puede reutilizar lógica (recomendado)*
- *Debe tener:*
  - *listado*
  - *useQuery*
  - *useMutation*
  - *manejo del modal*
  - *tanstack form*
  - *tanstack table*

IngredientsPage

- *Estructura similar a CategoriasPage*
- *Preparada para trabajar con ingredientes*
- *Puede reutilizar lógica (recomendado)*
- *Debe tener:*
    - *listado*
    - *useQuery*
    - *useMutation*
    - *manejo del modal*
    - *tanstack form*
    - *tanstack table*

g) Navegación

Implementar rutas usando React Router.

Instalar:

pnpm add react-router-dom

Configuración básica

Definir rutas:

- */categories → CategoriasPage*
- */products → ProductsPage*
- */ingredients→ IngredientesPage*

Navbar

Actualizar Navbar.tsx para incluir navegación:

- *Link a **Categorías***
- *Link a **Productos***
- *Link a **Productos***

h) App.tsx (cambio clave)

*Ahora **App ya no tiene lógica de negocio**.*

Solo debe encargarse de:

- *Definir rutas*
- *Renderizar el Navbar*
- *Mostrar las páginas*

Comportamiento esperado (actualizado)

- *La app inicia en /categories*
- *Se puede navegar a /products*
- *Se puede navegar a /ingredients*
- *Cada página maneja su propia lógica*
- *No hay lógica mezclada en App (orden profesional)*

HISTORIAS DE USUARIO

Las siguientes historias de usuario describen, desde la perspectiva del actor que opera el sistema, el comportamiento esperado de la aplicacion. Cada historia sigue el formato "Como... quiero... para..." y se acompania de sus criterios de aceptacion, que constituyen la condicion verificable de que la historia fue implementada correctamente.

ID	Historia de usuario y criterios de aceptacion	Canal de verificacion
HU-01	Listar categorias con tanStack Table Como administrador del catalogo, quiero ver todas las categorias en una tabla interactiva con ordenamiento y busqueda, para gestionar el inventario de forma eficiente. Criterios de aceptacion: • CategoriasPage usa useQuery para GET /categorias y renderiza con tanStack Table. • La tabla soporta ordenamiento por columnas (nombre, descripcion). • Si no hay categorias, se muestra un estado vacio. • Los errores de red se informan al usuario.	Frontend (React)
HU-02	Crear categoria con tanStack Form Como administrador, quiero crear una categoria usando un formulario con validacion tipada.	Frontend (React)

	Criterios de aceptacion: • El boton "Nueva categoria" abre un modal con tanStack Form y validacion. • useMutation ejecuta POST /categorias; al exito invalida la query y cierra el modal. • Error 409 (nombre duplicado) se muestra sin cerrar el modal.	
HU-03	Editar categoria Como administrador, quiero editar una categoria desde la tabla. Criterios de aceptacion: • El boton Editar abre el modal precargado con tanStack Form. • useMutation ejecuta PUT /categorias/{id}; al exito invalida la cache. • El modal diferencia alta de edicion en titulo y comportamiento.	Frontend (React)
HU-04	Eliminar categoria Como administrador, quiero eliminar categorias obsoletas. Criterios de aceptacion: • El boton Eliminar pide confirmacion y ejecuta useMutation con DELETE. • Si la categoria tiene productos, se muestra error 409. • Tras exito, la tabla se actualiza sin recargar.	Frontend (React)
HU-05	CRUD de Ingredientes Como administrador, quiero gestionar ingredientes con la misma interfaz que categorias. Criterios de aceptacion: • IngredientsPage tiene la misma estructura: useQuery, tanStack Table, tanStack Form, useMutation.	Frontend (React)

	• GET /ingredientes, POST, PUT/{id}, DELETE/{id} con validacion Pydantic. • El nombre de ingrediente es obligatorio y unico. • Navegacion a /ingredients desde el Navbar.	
HU-06	CRUD de Productos Como administrador, quiero gestionar productos con la misma interfaz. Criterios de aceptacion: • ProductsPage con useQuery, tanStack Table, tanStack Form, useMutation. • GET /productos, POST, PUT/{id}, DELETE/{id} con validacion Pydantic. • Productos incluyen nombre, descripcion, precio_base, ingredientes asociados.	Frontend (React)
HU-07	CRUD de Pedidos y DetallePedido (REST Client) Como administrador, quiero gestionar pedidos desde el backend. Criterios de aceptacion: • GET /pedido lista pedidos; GET /pedido/{id} devuelve detalle. • POST /pedido crea un pedido con sus detalles (201) o 422 si datos invalidos. • PUT /pedido/{id} actualiza estado; DELETE /pedido/{id} elimina. • Todas las operaciones se demuestran con archivos .http.	Backend (REST Client)

REGLAS DE NEGOCIO

Las reglas de negocio definen las restricciones que el sistema debe hacer cumplir para mantener la integridad de los datos. Son responsabilidad del backend (capa de validacion con Pydantic y logica de los servicios) y complementan las historias de usuario: una historia describe que quiere lograr el usuario, mientras que una regla de negocio establece que esta permitido y que debe rechazarse.

ID	Regla de negocio	Entidad / Verificacion
RN-01	Nombre de categoria obligatorio (min_length=1, sin solo espacios).	Categoria / 422
RN-02	Nombre de categoria unico (sin distincion de mayusculas).	Categoria / 409
RN-03	Descripcion de categoria opcional, default "".	Categoria / 201 o 200
RN-04	No operar sobre categoria inexistente.	Categoria / 404
RN-05	No eliminar categoria con productos asociados.	Categoria / 409
RN-06	Nombre de producto obligatorio, no unico.	Producto / 422
RN-07	precio_base >= 0, no negativos.	Producto / 422
RN-08	disponible es booleano, default True.	Producto / 201
RN-09	No operar sobre producto inexistente.	Producto / 404
RN-10	Nombre de ingrediente obligatorio (min_length=1, sin solo espacios) y unico.	Ingrediente / 422 o 409
RN-11	No operar sobre ingrediente inexistente.	Ingrediente / 404
RN-12	Un pedido debe tener al menos un DetallePedido para ser creado.	Pedido / 422
RN-13	No operar sobre pedido inexistente.	Pedido / 404
RN-14	El id de cada entidad es autoincremental, asignado por el backend.	Todas / id de solo lectura
RN-16	Toda peticion con datos invalidos o conflicto no modifica el estado y devuelve error.	Todas / 404, 409 o 422

HISTORIAS DE USUARIO

Las siguientes historias de usuario describen, desde la perspectiva del actor que opera el sistema, el comportamiento esperado de la aplicacion. Cada historia sigue el formato "Como... quiero... para..." y se acompania de sus criterios de aceptacion, que constituyen la condicion verificable de que la historia fue implementada correctamente.

ID	Historia de usuario y criterios de aceptacion	Canal de verificacion
HU-01	Listar categorias con tanStack Table Como administrador del catalogo, quiero ver todas las categorias en una tabla interactiva con ordenamiento y busqueda, para gestionar el inventario de forma eficiente. Criterios de aceptacion: • CategoriasPage usa useQuery para GET /categorias y renderiza con tanStack Table. • La tabla soporta ordenamiento por columnas (nombre, descripcion). • Si no hay categorias, se muestra un estado vacio. • Los errores de red se informan al usuario.	Frontend (React)
HU-02	Crear categoria con tanStack Form Como administrador, quiero crear una categoria usando un formulario con validacion tipada. Criterios de aceptacion: • El boton "Nueva categoria" abre un modal con tanStack Form y validacion. • useMutation ejecuta POST /categorias; al exito invalida la query y cierra el modal. • Error 409 (nombre duplicado) se muestra sin cerrar el modal.	Frontend (React)
HU-03	Editar categoria Como administrador, quiero editar una categoria desde la tabla. Criterios de aceptacion: • El boton Editar abre el modal precargado con tanStack Form.	Frontend (React)

	• useMutation ejecuta PUT /categorias/{id}; al exito invalida la cache. • El modal diferencia alta de edicion en titulo y comportamiento.	
HU-04	Eliminar categoria Como administrador, quiero eliminar categorias obsoletas. Criterios de aceptacion: • El boton Eliminar pide confirmacion y ejecuta useMutation con DELETE. • Si la categoria tiene productos, se muestra error 409. • Tras exito, la tabla se actualiza sin recargar.	Frontend (React)
HU-05	CRUD de Ingredientes Como administrador, quiero gestionar ingredientes con la misma interfaz que categorias. Criterios de aceptacion: • IngredientsPage tiene la misma estructura: useQuery, tanStack Table, tanStack Form, useMutation. • GET /ingredientes, POST, PUT/{id}, DELETE/{id} con validacion Pydantic. • El nombre de ingrediente es obligatorio y unico. • Navegacion a /ingredients desde el Navbar.	Frontend (React)
HU-06	CRUD de Productos Como administrador, quiero gestionar productos con la misma interfaz. Criterios de aceptacion: • ProductsPage con useQuery, tanStack Table, tanStack Form, useMutation. • GET /productos, POST, PUT/{id}, DELETE/{id} con validacion Pydantic.	Frontend (React)

	• Productos incluyen nombre, descripcion, precio_base, ingredientes asociados.	
HU-07	CRUD de Pedidos y DetallePedido (REST Client) Como administrador, quiero gestionar pedidos desde el backend. Criterios de aceptacion: • GET /pedido lista pedidos; GET /pedido/{id} devuelve detalle. • POST /pedido crea un pedido con sus detalles (201) o 422 si datos invalidos. • PUT /pedido/{id} actualiza estado; DELETE /pedido/{id} elimina. • Todas las operaciones se demuestran con archivos .http.	Backend (REST Client)

REGLAS DE NEGOCIO

Las reglas de negocio definen las restricciones que el sistema debe hacer cumplir para mantener la integridad de los datos. Son responsabilidad del backend (capa de validacion con Pydantic y logica de los servicios) y complementan las historias de usuario: una historia describe que quiere lograr el usuario, mientras que una regla de negocio establece que esta permitido y que debe rechazarse.

ID	Regla de negocio	Entidad / Verificacion
RN-01	Nombre de categoria obligatorio (min_length=1, sin solo espacios).	Categoria / 422
RN-02	Nombre de categoria unico (sin distincion de mayusculas).	Categoria / 409
RN-03	Descripcion de categoria opcional, default "".	Categoria / 201 o 200
RN-04	No operar sobre categoria inexistente.	Categoria / 404
RN-05	No eliminar categoria con productos asociados.	Categoria / 409
RN-06	Nombre de producto obligatorio, no unico.	Producto / 422

RN-07	precio_base >= 0, no negativos.	Producto / 422
RN-08	disponible es booleano, default True.	Producto / 201
RN-09	No operar sobre producto inexistente.	Producto / 404
RN-10	Nombre de ingrediente obligatorio (min_length=1, sin solo espacios) y unico.	Ingrediente / 422 o 409
RN-11	No operar sobre ingrediente inexistente.	Ingrediente / 404
RN-12	Un pedido debe tener al menos un DetallePedido para ser creado.	Pedido / 422
RN-13	No operar sobre pedido inexistente.	Pedido / 404
RN-14	El id de cada entidad es autoincremental, asignado por el backend.	Todas / id de solo lectura
RN-15	Los datos se mantienen en memoria; al reiniciar, se pierden.	Todas / comportamiento esperado
RN-16	Toda peticion con datos invalidos o conflicto no modifica el estado y devuelve error.	Todas / 404, 409 o 422

HISTORIAS DE USUARIO

Las siguientes historias de usuario describen, desde la perspectiva del actor que opera el sistema, el comportamiento esperado de la aplicacion. Cada historia sigue el formato "Como... quiero... para..." y se acompania de sus criterios de aceptacion, que constituyen la condicion verificable de que la historia fue implementada correctamente.

ID	Historia de usuario y criterios de aceptacion	Canal de verificacion
HU-01	Listar categorias con tanStack Table Como administrador del catalogo, quiero ver todas las categorias en una tabla interactiva con ordenamiento y busqueda, para gestionar el inventario de forma eficiente. Criterios de aceptacion:	Frontend (React)

	• CategoriasPage usa useQuery para GET /categorias y renderiza con tanStack Table. • La tabla soporta ordenamiento por columnas (nombre, descripcion). • Si no hay categorias, se muestra un estado vacio. • Los errores de red se informan al usuario.	
HU-02	Crear categoria con tanStack Form Como administrador, quiero crear una categoria usando un formulario con validacion tipada. Criterios de aceptacion: • El boton "Nueva categoria" abre un modal con tanStack Form y validacion. • useMutation ejecuta POST /categorias; al exito invalida la query y cierra el modal. • Error 409 (nombre duplicado) se muestra sin cerrar el modal.	Frontend (React)
HU-03	Editar categoria Como administrador, quiero editar una categoria desde la tabla. Criterios de aceptacion: • El boton Editar abre el modal precargado con tanStack Form. • useMutation ejecuta PUT /categorias/{id}; al exito invalida la cache. • El modal diferencia alta de edicion en titulo y comportamiento.	Frontend (React)
HU-04	Eliminar categoria Como administrador, quiero eliminar categorias obsoletas. Criterios de aceptacion: • El boton Eliminar pide confirmacion y ejecuta useMutation con DELETE.	Frontend (React)

	• Si la categoria tiene productos, se muestra error 409. • Tras exito, la tabla se actualiza sin recargar.	
HU-05	CRUD de Ingredientes Como administrador, quiero gestionar ingredientes con la misma interfaz que categorias. Criterios de aceptacion: • IngredientsPage tiene la misma estructura: useQuery, tanStack Table, tanStack Form, useMutation. • GET /ingredientes, POST, PUT/{id}, DELETE/{id} con validacion Pydantic. • El nombre de ingrediente es obligatorio y unico. • Navegacion a /ingredients desde el Navbar.	Frontend (React)
HU-06	CRUD de Productos Como administrador, quiero gestionar productos con la misma interfaz. Criterios de aceptacion: • ProductsPage con useQuery, tanStack Table, tanStack Form, useMutation. • GET /productos, POST, PUT/{id}, DELETE/{id} con validacion Pydantic. • Productos incluyen nombre, descripcion, precio_base, ingredientes asociados.	Frontend (React)
HU-07	CRUD de Pedidos y DetallePedido (REST Client) Como administrador, quiero gestionar pedidos desde el backend. Criterios de aceptacion: • GET /pedido lista pedidos; GET /pedido/{id} devuelve detalle. • POST /pedido crea un pedido con sus detalles (201) o 422 si datos invalidos.	Backend (REST Client)

	• PUT /pedido/{id} actualiza estado; DELETE /pedido/{id} elimina. • Todas las operaciones se demuestran con archivos .http.	

REGLAS DE NEGOCIO

Las reglas de negocio definen las restricciones que el sistema debe hacer cumplir para mantener la integridad de los datos. Son responsabilidad del backend (capa de validacion con Pydantic y logica de los servicios) y complementan las historias de usuario: una historia describe que quiere lograr el usuario, mientras que una regla de negocio establece que esta permitido y que debe rechazarse.

ID	Regla de negocio	Entidad / Verificacion
RN-01	Nombre de categoria obligatorio (min_length=1, sin solo espacios).	Categoria / 422
RN-02	Nombre de categoria unico (sin distincion de mayusculas).	Categoria / 409
RN-03	Descripcion de categoria opcional, default "".	Categoria / 201 o 200
RN-04	No operar sobre categoria inexistente.	Categoria / 404
RN-05	No eliminar categoria con productos asociados.	Categoria / 409
RN-06	Nombre de producto obligatorio, no unico.	Producto / 422
RN-07	precio_base >= 0, no negativos.	Producto / 422
RN-08	disponible es booleano, default True.	Producto / 201
RN-09	No operar sobre producto inexistente.	Producto / 404
RN-10	Nombre de ingrediente obligatorio (min_length=1, sin solo espacios) y unico.	Ingrediente / 422 o 409
RN-11	No operar sobre ingrediente inexistente.	Ingrediente / 404

RN-12	Un pedido debe tener al menos un DetallePedido para ser creado.	Pedido / 422
RN-13	No operar sobre pedido inexistente.	Pedido / 404
RN-14	El id de cada entidad es autoincremental, asignado por el backend.	Todas / id de solo lectura
RN-15	Los datos se mantienen en memoria; al reiniciar, se pierden.	Todas / comportamiento esperado
RN-16	Toda peticion con datos invalidos o conflicto no modifica el estado y devuelve error.	Todas / 404, 409 o 422

CONCLUSIONES ESPERADAS

Al finalizar el trabajo práctico, el estudiante debe demostrar:

- **Integración Full Stack:** El frontend React consume la API FastAPI mediante fetch nativo. El CRUD de Categorías funciona de extremo a extremo.
- **Modal con useState:** El formulario de alta y edición está dentro de un modal controlado con estado booleano.
- **Hooks aplicados correctamente:** useState y useEffect utilizados en App.tsx. Los demás componentes son funcionales puros sin hooks.
- **Tipado TypeScript:** Interfaces Categoria y Producto definidas. Props tipados en todos los componentes .tsx.
- **Backend REST documentado:** Endpoints de Pedido y DetallePedido funcionando y demostrados en el archivo .http.
- **CORS configurado:** El backend permite peticiones desde el origen del frontend.

RÚBRICA DE EVALUACIÓN

Criterio	Descripción	Puntos
CRUD Categories – Frontend	Listar, crear, editar y eliminar categorías desde React usando useQuery, tanstackForm, tanstackTable.	10
CRUD Ingredients – Frontend	Listar, crear, editar y eliminar categorías desde React usando useQuery, tanstackForm, tanstackTable	10
CRUD Products– Frontend	Listar, crear, editar y eliminar categorías desde React usando useQuery, tanstackForm, tanstackTable	10
Modal funcional	Modal que se abre para crear y editar categorías, controlado con useState. Se cierra al guardar o cancelar.	15
CRUD Pedidos– Backend	Endpoints FastAPI GET /pedido, POST, PUT/{id}, DELETE/{id} con Pydantic y persistencia en lista/DB.	20
CRUD Pedidos y relación – REST Client	Endpoints de Pedido y DetallePedido demostrados con REST Client (.http). No requiere UI.	15
Tipado TypeScript	Interfaces Categoria y Producto tipadas. Props tipados en todos los componentes.	10
Estructura y buenas prácticas	Carpetas components/ y types/. Tailwind CSS aplicado. Sin errores de TS en consola.	10

Modalidad de Entrega

- Eliminar la carpeta node_modules antes de comprimir
- Proyecto Frontend comprimido en un archivo .zip
- Eliminar .venv antes de comprimir
- Proyecto Backend comprimido en un archivo .zip

RECURSOS ADICIONALES

 Documentación oficial:

- React: https://react.dev/
- TypeScript: https://www.typescriptlang.org/docs/
- Tailwind CSS: https://tailwindcss.com/docs/
- Vite: https://vitejs.dev/
- FastAPI: https://fastapi.tiangolo.com/